

Lab Practice Problem 10A

Filename: No File Submission

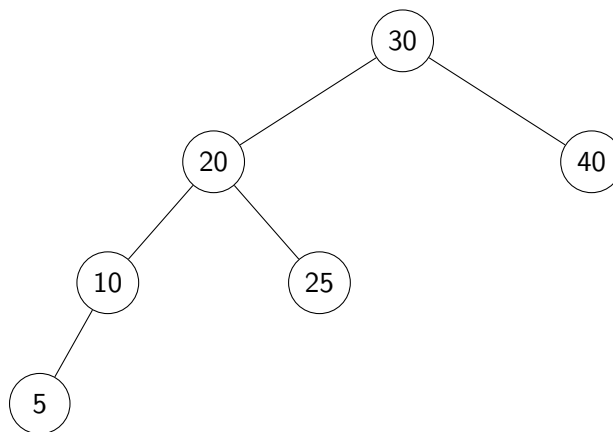
In this activity, you will work with AVL trees by determining whether given trees satisfy the AVL property, as well as performing insertion and deletion operations. The goal of this exercise is to strengthen your understanding of AVL tree properties, balanced tree behavior, and self-balancing tree operations.

Important: Do not run any program or use code to compute the results for this activity. The purpose of this exercise is to practice tracing the algorithm manually. For consistency, compute each balance factor as the height of the left subtree minus the height of the right subtree. If a child is missing (i.e., NULL), assume its height is -1.

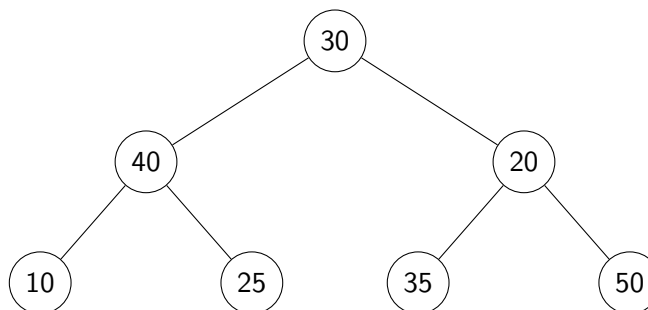
AVL Trees

Determine whether each of the given trees is an AVL tree. For each tree, state Yes or No and briefly justify your answer.

Tree 1



Tree 2



AVL Insert Operation

Insert the given values into an initially empty AVL tree in the specified order. After each insertion, draw the resulting tree. Then, determine the height of the final tree.

Hint: Insertion in an AVL tree is similar to insertion in a BST. The only additional step is to rebalance the tree when necessary. Moreover, at most two rotations are needed to restore balance after a single insertion.

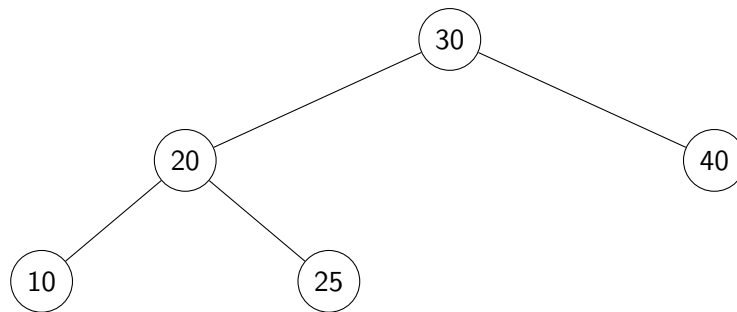
30, 20, 10, 40, 50, 5, 7, 45, 43

AVL Remove Operation

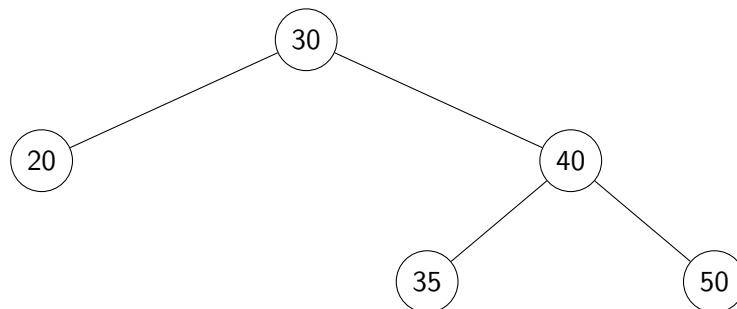
Given each AVL tree below, delete the specified node and draw the resulting AVL tree after rebalancing. Assume that deletion follows the delete-by-copying method. When applicable, replace the deleted node with its successor. Then, determine the height of the final tree.

Hint: Deletion in an AVL tree is similar to deletion in a BST. The only additional step is to rebalance the tree when necessary. Unlike insertion, there is no guaranteed fixed number of rotations that will restore balance. So, you must continue checking and rebalancing all the way up to the root if needed.

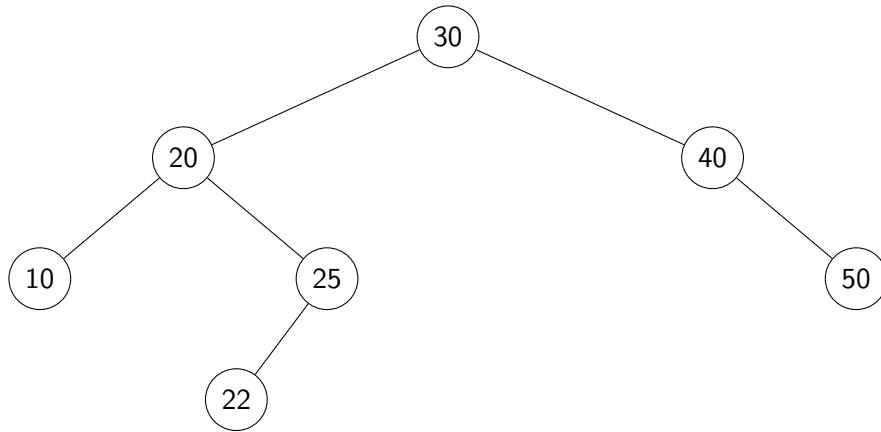
Delete Node 40



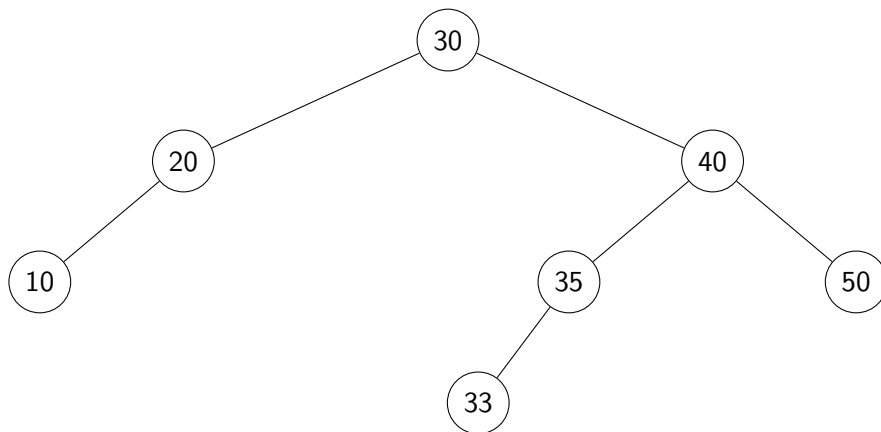
Delete Node 20



Delete Node 50



Delete Node 10



Algorithms

The following pseudocode shows how to rebalance an AVL tree when an invalid balance factor is detected.

LL or LR Configurations

```
If BF of node x is +2, left subtree is longer (problematic)
    Check the sign of the BF of left child y of node x
        If + then LL, rotate right at x
        If - then LR, rotate left at y then right at x

    If 0 then LL, rotate right at x (deletion only)
```

RR or RL Configurations

```
If BF of node x is -2, right subtree is longer (problematic)
    Check the sign of the BF of right child y of node x
        If - then RR, rotate left at x
        If + then RL, rotate right at y then left at x

    If 0 then RR, rotate left at x (deletion only)
```

Hint: A useful strategy is to examine the sign of the balance factor (BF). Recall that rebalancing should be performed immediately when a node's balance factor becomes invalid (i.e., +2 or -2).